Login | Protect | Data | kivy | Security | Attack | Firewall | Firebase

LASER SECURITY GATE

Protecting Guaranty

Group “1"

Embedded System

OUR PURPOSE

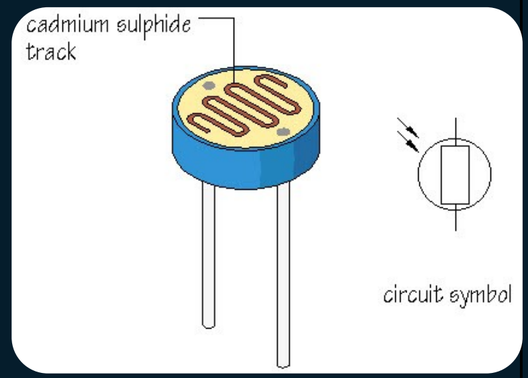
Our project is a security gate system providing a reliable and efficient method for detecting unauthorized access or breaches in secured areas. By using a laser beam as an invisible barrier, the system ensures that any interruption of the beam triggers an immediate buzzer alert, this system is designed to enhance security in residential, commercial, and public settings by providing real-time intrusion detection, minimizing risks, and reinforcing overall safety.

MAIN HARDWARE COMPONENTS

Keypad

Breadboard

LDR

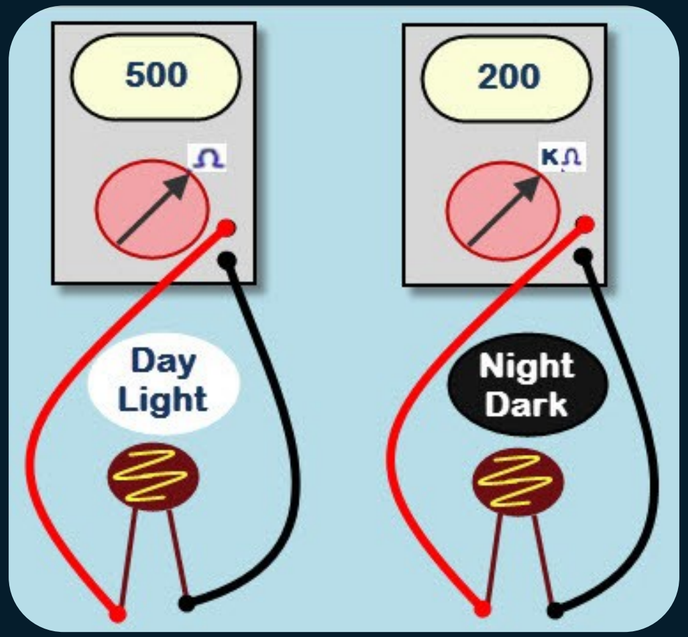


ESP-32S

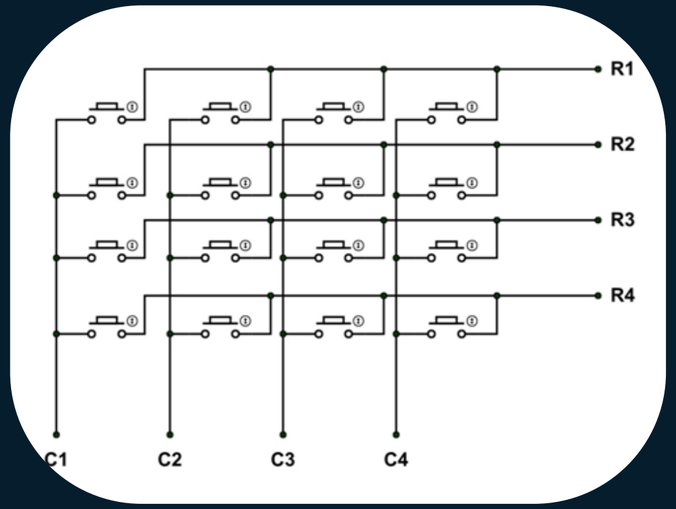
Buzzer

LED

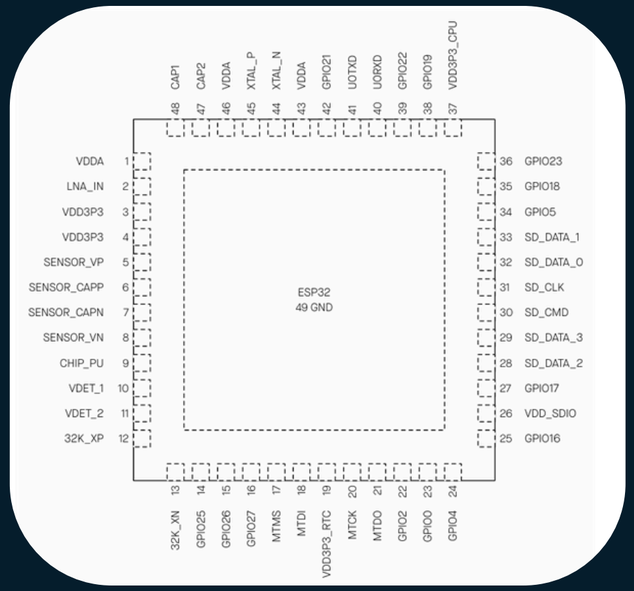
Datasheets



LDR



Keypad internal structure



ESP-32S

DESCRIPTION

Buzzer with continuous tone generator with a diameter of 12 mm. Power voltage of 5 V.

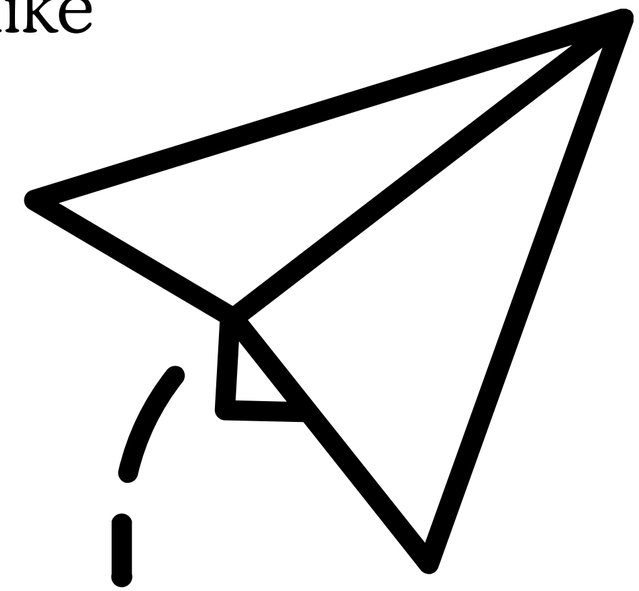
Specifications:

- Supply voltage: 5 V
- Volume 85 dB
- Current consumption: 30 mA max.
- Frequency: 2.3 kHz ± 500 Hz
- Diameter: 12 mm

Buzzer

BUAD RATE IN ESP-32

- On the ESP32, 115200 is a commonly used baud rate for serial communication, just like 9600, but it offers faster data transfer.



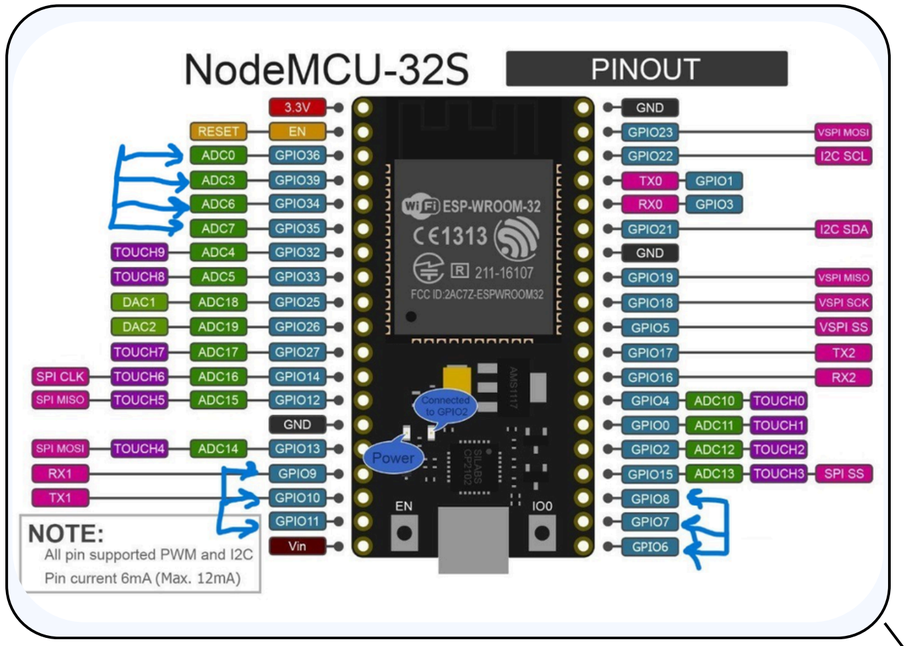
CONNECTIONS

Connecting the Buzzer (Small Buzzer 5V):

- The positive terminal of the buzzer to a digital GPIO 27.
- The negative terminal to GND.

Connecting the Photo Resistor (LDR):

- One leg of the photo resistor to a 510-ohm resistor, and the other leg of the resistor to VCC (3.3V).
- The other leg of the resistor is connected to the ground (GND).
- The other leg of the photo resistor to an ADC (Analog-to Digital Converter) pin on the ESP32 (GPIO 33).



CONNECTIONS

Connecting the LASER (5V):

- One leg of the Laser connected to the ground (GND).
- The other leg to an ADC (Analog-to-Digital Converter) pin on the ESP32 GPIO 25.

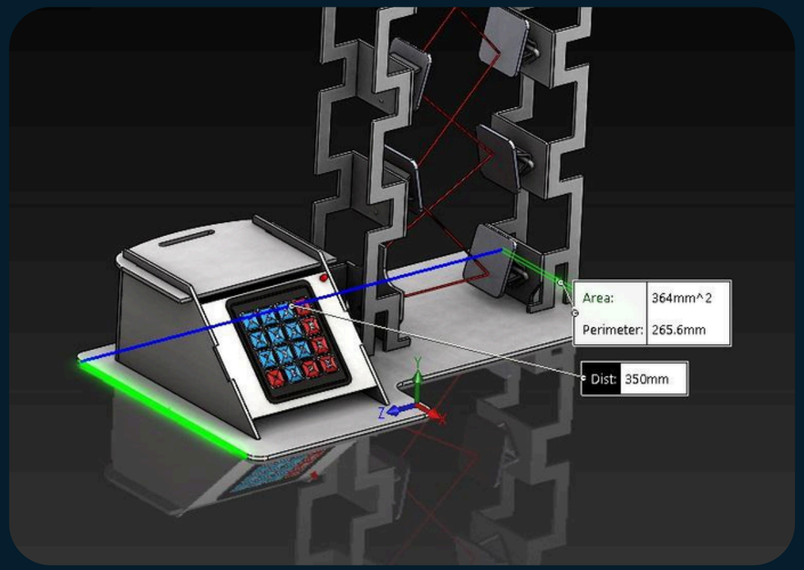
Connecting LED :

- Connecting the (LED):
- One leg of the LED to a 510-ohm resistor,
- The other leg of the resistor is connected to the ground (GND).
- The other leg of the LED to ADC (Analog-to Digital Converter) pin on the ESP32 (P22).

Connecting the Matrix Keypad (16 Keys):

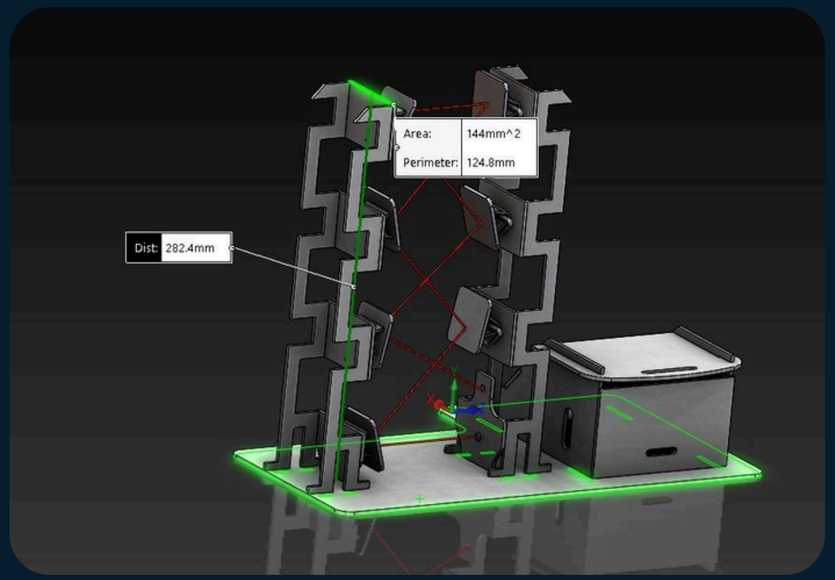
- The keypad has 8 pins (4 rows and 4 columns).
- Connect these pins to the GPIO pins of the ESP32: Connect them to GPIO pins
  - rows connect to GPIO (18,5,17,16)
  - columns connect to GPIO (4,0,2,15)

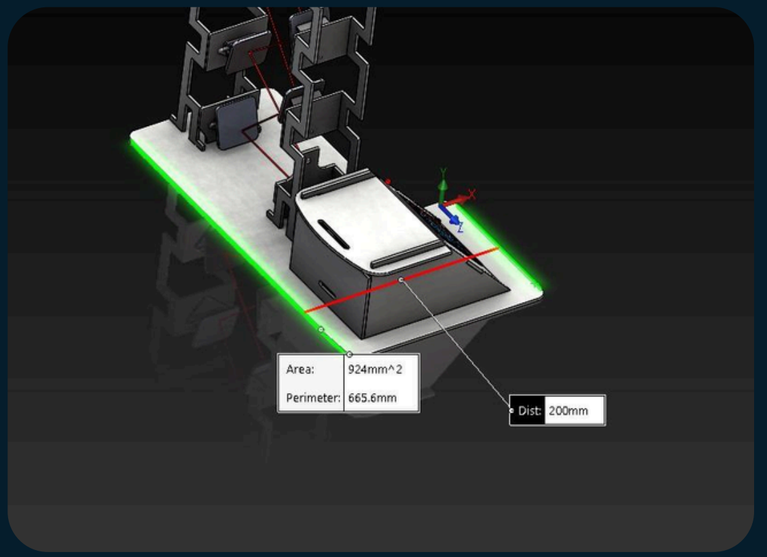
3-D PROTOTYPE DESIGN



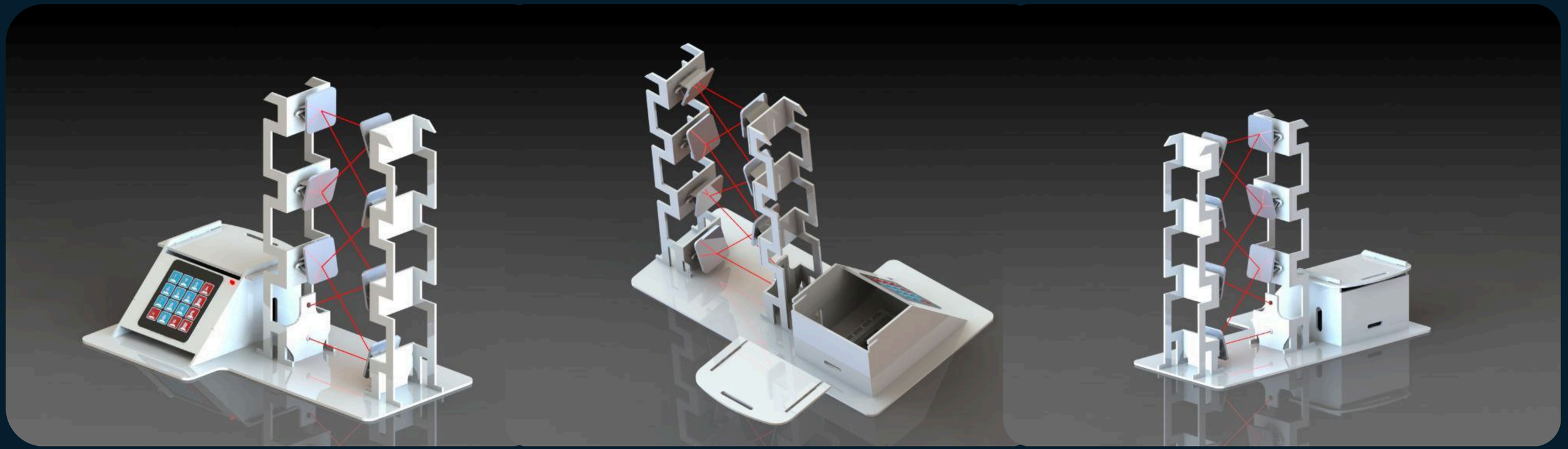
length: 350mm

height: 285mm





Width: 200mm



- This Design made on “ SOLID WORK ” application.
- It composite of 18 part.

SOFTWARE C CODE

System Design

Hardware Setup:

- **Laser Pin (25)**: Controls the laser that could be part of a security gate system.
- **Buzzer Pin (27):** Used for sound alerts (for wrong password or security breach).
- **LED Pin (22):** Provides visual feedback (lights up on wrong attempts or errors).
- **LDR Pin (33):** Reads light intensity to check for security breaches or certain conditions.
- **Keypad (4x4 matrix):** Allows users to enter a password for system access.

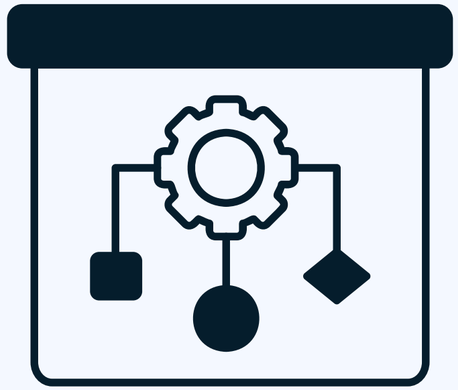
Security Features:

- The system locks or unlocks a gate by controlling the laser (turns on/off).
- The buzzer sounds when there is an incorrect password input or when maximum login attempts are exceeded.
- The system also uses an LDR to check for any light changes, simulating an alert when a gate is potentially open.

Variables for Password and System State

- Attempts: Tracks the number of failed login attempts.
- Data: Stores the password input by the user.
- Master: The predefined correct password ("1234")

Logic Flow



1. Password Authentication:

- The user enters a password through the keypad.
- If the entered password matches the Master password, the system will:
  - Turn off the laser (open the gate).
  - Reset any incorrect attempts.
- If the password is wrong:
  - The buzzer sounds.
  - The system tracks attempts and locks access after a set number of failed attempts (3).

2. Light Detection (LDR Check):

- The LDR detects if there is enough light.
- If light intensity drops below a certain threshold, the buzzer is triggered, and an alert is sent.

3. Handling Multiple Attempts:

- The system allows three attempts.
- If the wrong password is entered three times, the system locks access and sounds the buzzer to indicate the maximum attempts have been reached.

Core Functions

1. LDR Check (ldrCheck):

- This function reads the value from the LDR and compares it with a threshold. If the value is below the threshold (indicating low light, perhaps because a gate has been opened), the system will trigger the buzzer and LED.

2.Password Checking (passwordCheck):

- This function checks if the user has entered the correct password. If the entered password matches the predefined "Master" password, it calls the Correct() function to unlock the system. If not, it calls the InCorrect() function to indicate a failed attempt.

3. Handling Incorrect Passwords (InCorrect):

- If the user enters an incorrect password, this function increments the attempt counter and sounds the buzzer. If the number of attempts exceeds the allowed limit (max_attempts), it triggers the highAttempts() function to lock the system.

4. Handling Correct Password (Correct):

- When the correct password is entered, the laser is turned off (simulating opening the gate), and the system resets the attempts count.

5. Handling Maximum Attempts (highAttempts):

- If the user exceeds the maximum number of incorrect attempts, this function will sound the buzzer and keep it on for a period, indicating that access is denied.

Keypad Input: The loop constantly checks for keypresses on the keypad. When a key is pressed, it adds the character to the Data string

LDR Check:
The loop also continuously checks the LDR value to see if any significant change in light intensity occurs.

Password Validation:
After each keypress, the passwordCheck() function checks if the entered data matches the predefined password.

APP UI/UX

- This interface, created using figma, contains three buttons (ON, OFF, and B) with a simple and user-friendly layout. Here's how each button works, with the B button (buzzer) toggling between ON and OFF states.

Button Functionality:

1. ON Button:

- When pressed, it activates the laser (Gate ON).

2. OFF Button:

- When pressed, it deactivates the laser (Gate OFF).

3. B Button (Buzzer Toggle):

- This button alternates between ON and OFF states for the buzzer.

When pressed:

- If the buzzer is OFF, it turns it ON and updates the notification to show "Buzzer ON."
- If the buzzer is ON, it turns it OFF and updates the notification to show "Buzzer OFF."

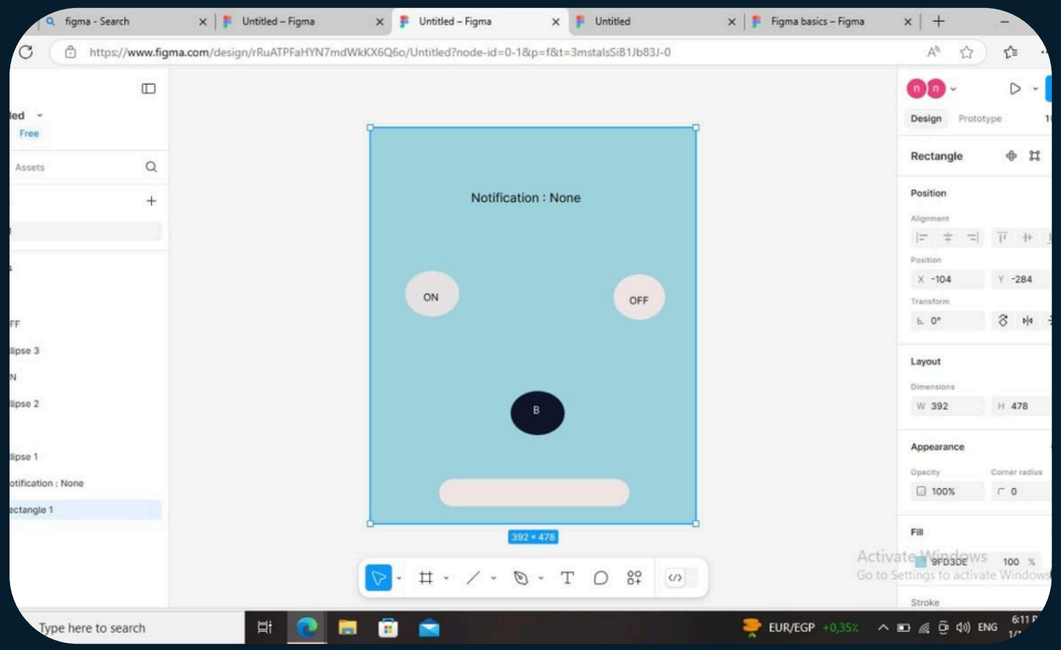
...Continued

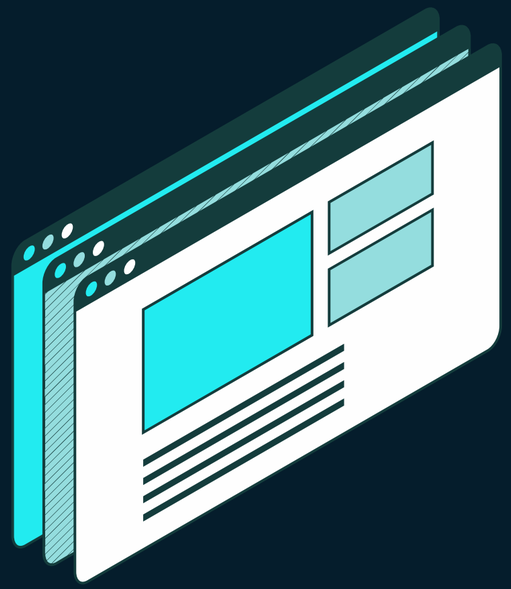
Color Scheme:

- **Light Blue Background:**
  A calm and clean color to ensure clear visibility of elements.

- **White Buttons (ON and OFF):**
  Provide strong contrast with the background and focus on the laser controls.

- **Dark Blue Buzzer Button (B):**
  Draws attention to its dual functionality with white text for clarity



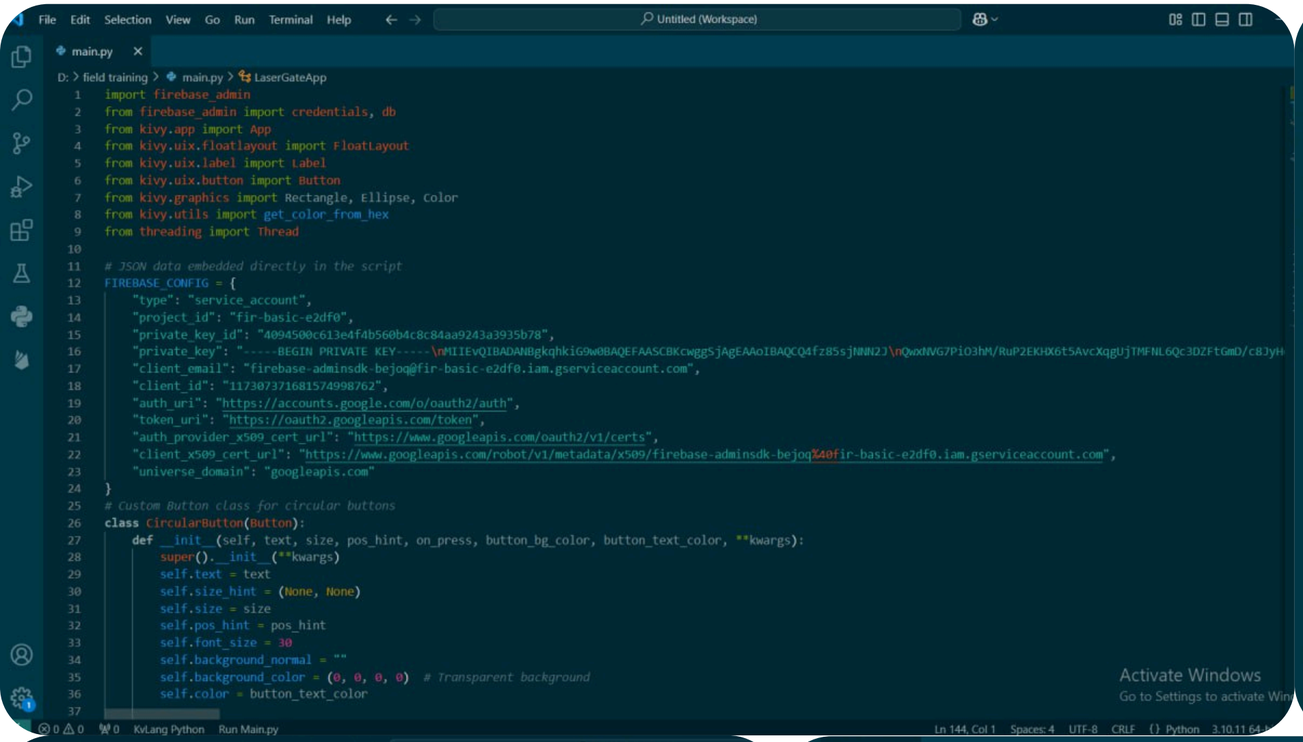


APP with Kivy

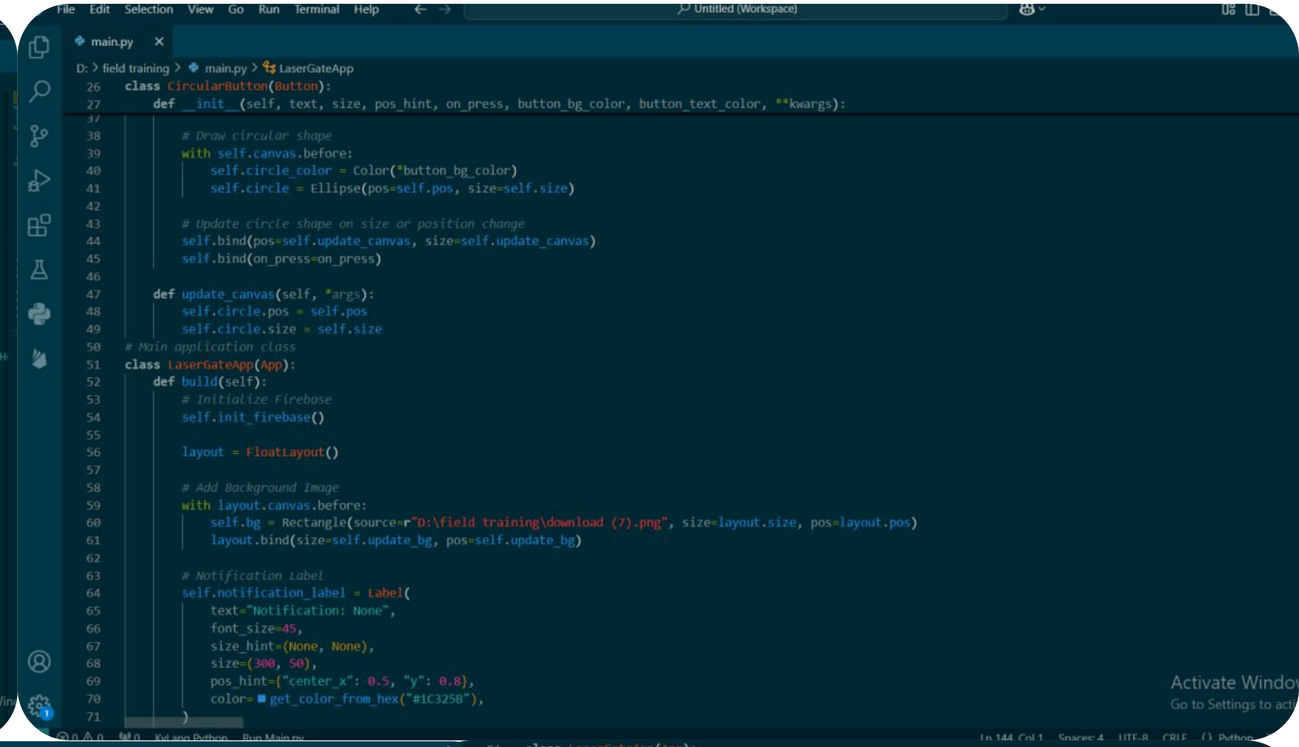
- The Laser Gate System application interface is designed to provide a **seamless** and **user-friendly** experience, aiming to simplify control over the laser gate system.

- The interface features three main buttons: **an (ON) button** to activate the laser, an **(OFF) button** to deactivate it, and a button for **gate state** if gate (ON/OFF), with a clear design ensuring easy access to these functions.
- Additionally, real-time notifications are sent to the user to keep them constantly updated on the system's status, enhancing both security and control.

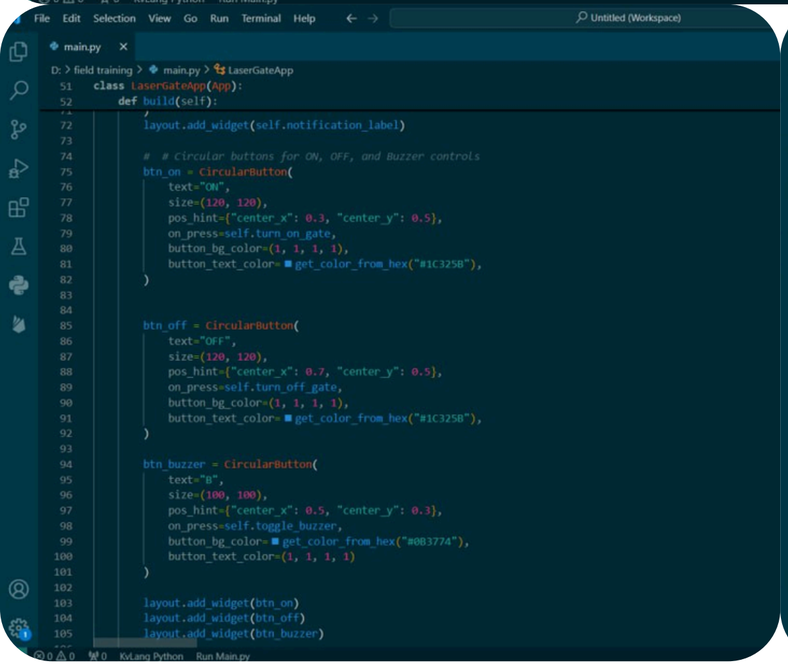
- The interface also includes **a (B) button for the buzzer**, which emits an alert sound in case of unusual activity, such as multiple incorrect password attempts or a hacking attempt. This feature enhances **system security** by alerting users to potential threats immediately.
- The overall design of the application is simple yet elegant, utilizing a harmonious color scheme and logically arranged interface elements to ensure a smooth user experience.



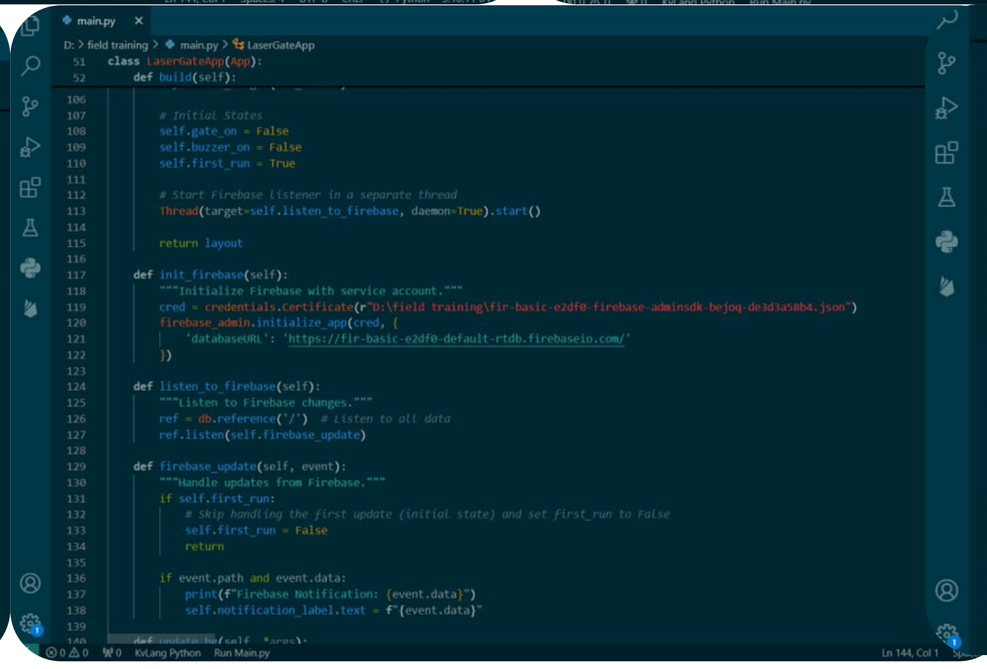
```python
import firebase_admin
from firebase_admin import credentials, db
from kivy.app import App
from kivy.uix.floatlayout import FloatLayout
from kivy.uix.label import Label
from kivy.uix.button import Button
from kivy.graphics import Rectangle, Ellipse, Color
from kivy.utils import get_color_from_hex
from threading import Thread

# JSON data embedded directly in the script
FIREBASE_CONFIG = {
    "type": "service_account",
    "project_id": "fir-basic-e2df0",
    "private_key_id": "4094500c613e4f4b560b4c8c84aa9243a3935b78",
    "private_key": "-----BEGIN PRIVATE KEY-----\nMIIEvQIBADANBgkqhkiG9w0BAQEFAASCBKcwggSjAgEAAoIBAQCQ4fz85sjNWN2J\nQwxNVG7PiO3hM/RuP2EKHX6t5AvcXqgUjTMFNL6Qc3DZFtGmD/c8JyH
    "client_email": "firebase-adminsdk-bejoq@fir-basic-e2df0.iam.gserviceaccount.com",
    "client_id": "117307371681574998762",
    "auth_uri": "https://accounts.google.com/o/oauth2/auth",
    "token_uri": "https://oauth2.googleapis.com/token",
    "auth_provider_x509_cert_url": "https://www.googleapis.com/oauth2/v1/certs",
    "client_x509_cert_url": "https://www.googleapis.com/robot/v1/metadata/x509/firebase-adminsdk-bejoq%40fir-basic-e2df0.iam.gserviceaccount.com",
    "universe_domain": "googleapis.com"
}
# Custom Button class for circular buttons
class CircularButton(Button):
    def __init__(self, text, size, pos_hint, on_press, button_bg_color, button_text_color, **kwargs):
        super().__init__(**kwargs)
        self.text = text
        self.size_hint = (None, None)
        self.size = size
        self.pos_hint = pos_hint
        self.font_size = 30
        self.background_normal = ""
        self.background_color = (0, 0, 0, 0)  # Transparent background
        self.color = button_text_color
```

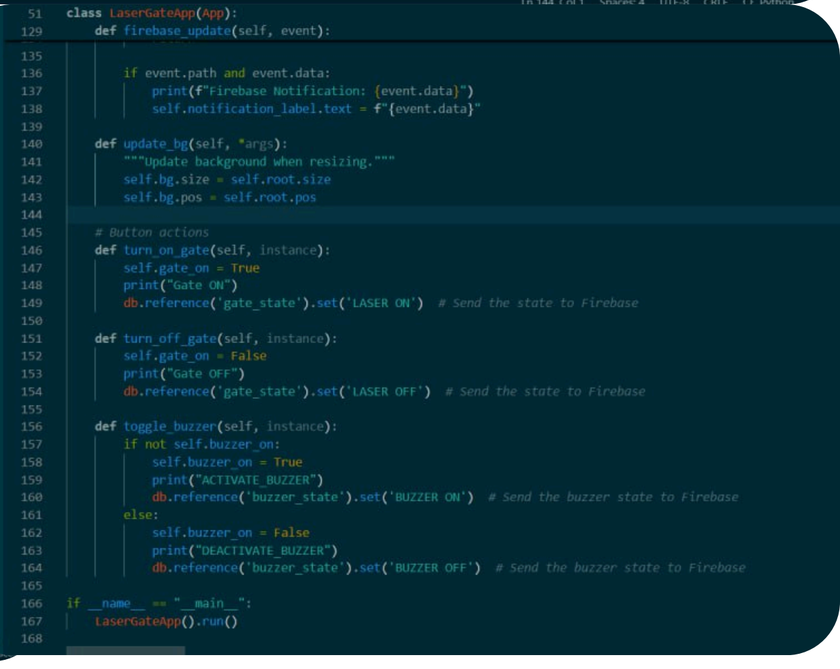
```python
class CircularButton(Button):
    def __init__(self, text, size, pos_hint, on_press, button_bg_color, button_text_color, **kwargs):

        # Draw circular shape
        with self.canvas.before:
            self.circle_color = Color(*button_bg_color)
            self.circle = Ellipse(pos=self.pos, size=self.size)

        # Update circle shape on size or position change
        self.bind(pos=self.update_canvas, size=self.update_canvas)
        self.bind(on_press=on_press)

    def update_canvas(self, *args):
        self.circle.pos = self.pos
        self.circle.size = self.size
# Main application class
class LaserGateApp(App):
    def build(self):
        # Initialize Firebase
        self.init_firebase()

        layout = FloatLayout()

        # Add Background Image
        with layout.canvas.before:
            self.bg = Rectangle(source=r"D:\field training\download (7).png", size=layout.size, pos=layout.pos)
            layout.bind(size=self.update_bg, pos=self.update_bg)

        # Notification Label
        self.notification_label = Label(
            text="Notification: None",
            font_size=45,
            size_hint=(None, None),
            size=(300, 50),
            pos_hint={"center_x": 0.5, "y": 0.8},
            color=■get_color_from_hex("#1C325B"),
        )
```

```python
class LaserGateApp(App):
    def build(self):
        layout.add_widget(self.notification_label)

        #  # Circular buttons for ON, OFF, and Buzzer controls
        btn_on = CircularButton(
            text="ON",
            size=(120, 120),
            pos_hint={"center_x": 0.3, "center_y": 0.5},
            on_press=self.turn_on_gate,
            button_bg_color=(1, 1, 1, 1),
            button_text_color=■get_color_from_hex("#1C325B"),
        )


        btn_off = CircularButton(
            text="OFF",
            size=(120, 120),
            pos_hint={"center_x": 0.7, "center_y": 0.5},
            on_press=self.turn_off_gate,
            button_bg_color=(1, 1, 1, 1),
            button_text_color=■get_color_from_hex("#1C325B"),
        )

        btn_buzzer = CircularButton(
            text="B",
            size=(100, 100),
            pos_hint={"center_x": 0.5, "center_y": 0.3},
            on_press=self.toggle_buzzer,
            button_bg_color=■get_color_from_hex("#0B3774"),
            button_text_color=(1, 1, 1, 1)
        )

        layout.add_widget(btn_on)
        layout.add_widget(btn_off)
        layout.add_widget(btn_buzzer)
```

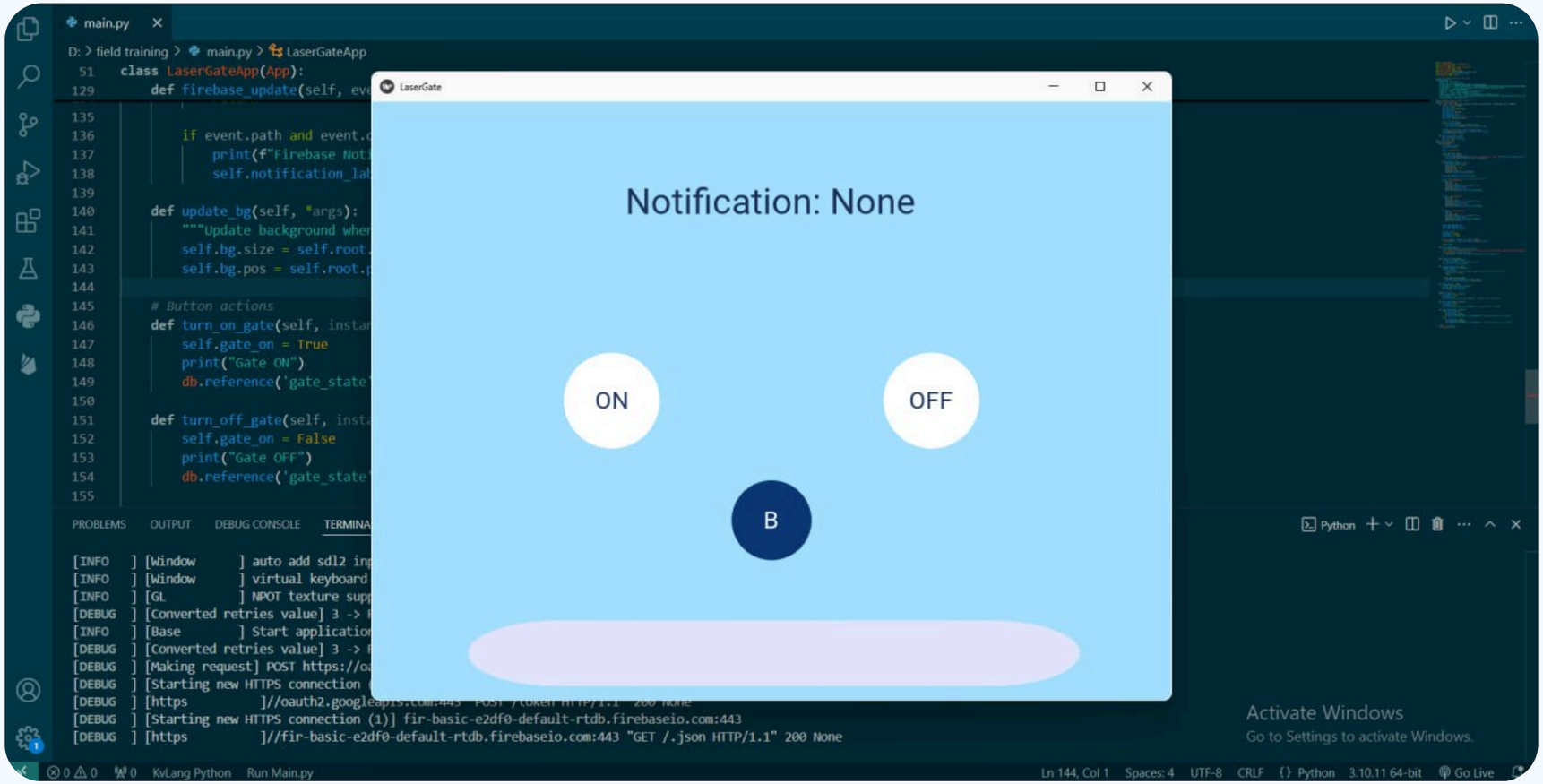


```python
class LaserGateApp(App):
    def build(self):

        # Initial States
        self.gate_on = False
        self.buzzer_on = False
        self.first_run = True

        # Start Firebase listener in a separate thread
        Thread(target=self.listen_to_firebase, daemon=True).start()

        return layout

    def init_firebase(self):
        """Initialize Firebase with service account."""
        cred = credentials.Certificate(r"D:\field training\fir-basic-e2df0-firebase-adminsdk-bejoq-de3d3a58b4.json")
        firebase_admin.initialize_app(cred, {
            'databaseURL': 'https://fir-basic-e2df0-default-rtdb.firebaseio.com/'
        })

    def listen_to_firebase(self):
        """Listen to Firebase changes."""
        ref = db.reference('/')  # Listen to all data
        ref.listen(self.firebase_update)

    def firebase_update(self, event):
        """Handle updates from Firebase."""
        if self.first_run:
            # Skip handling the first update (initial state) and set first_run to False
            self.first_run = False
            return

        if event.path and event.data:
            print(f"Firebase Notification: {event.data}")
            self.notification_label.text = f"{event.data}"

    def update_bg(self, *args):
```



```python
class LaserGateApp(App):
    def firebase_update(self, event):

        if event.path and event.data:
            print(f"Firebase Notification: {event.data}")
            self.notification_label.text = f"{event.data}"

    def update_bg(self, *args):
        """Update background when resizing."""
        self.bg.size = self.root.size
        self.bg.pos = self.root.pos

    # Button actions
    def turn_on_gate(self, instance):
        self.gate_on = True
        print("Gate ON")
        db.reference('gate_state').set('LASER ON')  # Send the state to Firebase

    def turn_off_gate(self, instance):
        self.gate_on = False
        print("Gate OFF")
        db.reference('gate_state').set('LASER OFF')  # Send the state to Firebase

    def toggle_buzzer(self, instance):
        if not self.buzzer_on:
            self.buzzer_on = True
            print("ACTIVATE_BUZZER")
            db.reference('buzzer_state').set('BUZZER ON')  # Send the buzzer state to Firebase
        else:
            self.buzzer_on = False
            print("DEACTIVATE_BUZZER")
            db.reference('buzzer_state').set('BUZZER OFF')  # Send the buzzer state to Firebase

if __name__ == "__main__":
    LaserGateApp().run()
```

Code Run


main.py
D: > field training > main.py > LaserGateApp
class LaserGateApp(App):
def firebase_update(self, ev
if event.path and event.
print(f"Firebase Not
self.notification_la
def update_bg(self, *args):
"""Update background whe
self.bg.size = self.root.
self.bg.pos = self.root.
# Button actions
def turn_on_gate(self, insta
self.gate_on = True
print("Gate ON")
db.reference('gate_state
def turn_off_gate(self, inst
self.gate_on = False
print("Gate OFF")
db.reference('gate_state
PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINA
Python
[INFO] [Window] auto add sdl2 in
[INFO] [Window] virtual keyboard
[INFO] [GL] NPOT texture sup
[DEBUG] [Converted retries value] 3 ->
[INFO] [Base] Start applicatio
[DEBUG] [Converted retries value] 3 ->
[DEBUG] [Making request] POST https://o
[DEBUG] [Starting new HTTPS connection
[DEBUG] [https]//oauth2.googleapis.com:443 "POST /token HTTP/1.1" 200 None
[DEBUG] [Starting new HTTPS connection (1)] fir-basic-e2df0-default-rtdb.firebaseio.com:443
[DEBUG] [https]//fir-basic-e2df0-default-rtdb.firebaseio.com:443 "GET /.json HTTP/1.1" 200 None
LaserGate
Notification: None
ON
OFF
B
Activate Windows
Go to Settings to activate Windows.
KvLang Python
Run Main.py
Ln 144, Col 1
Spaces: 4
UTF-8
CRLF
Python
3.10.11 64-bit
Go Live

Google Colab

Google Colab is a cloud-based integrated development environment (IDE) that allows you to run Python code on remote servers without the need for local setup.

Some Commands in Google Colab:

1. !pip install buildozer.
2. !buildozer init.
3. !buildozer android debug deploy run.

- These commands set up the environment and use Buildozer to turn a Python project into an Android app, allowing you to run and test it on Android devices.

Buildozer:

- Buildozer is an open-source tool used to convert Python applications into executable files that can run on multiple platforms such as Android, iOS, Linux, and Windows.

The buildozer.spec file

...Google Colab

The buildozer.spec file:

- When using Buildozer, a file called buildozer.spec is created in the project folder. This file defines how to build the app and configure the Buildozer environment for the app.

How to create a buildozer.spec:

- You can create this file by running the following command buildozer init

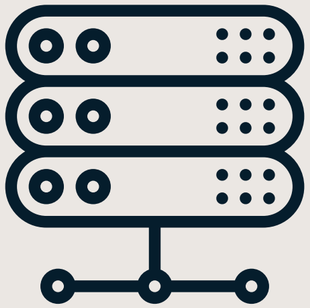
How to customize the buildozer.spec based on your needs:

- **Customize the app:** Edit settings like title, version, package name, requirements
- **Add libraries:** In requirements, you can specify external libraries your app needs.
- **Permissions:** In the [android] section, you can specify permissions the app requires, such as INTERNET.
- **Testing the app:** After the build is complete, you can test the app on an Android device by using the generated APK file.

- **What is an API?**

Users

WiFi Connection



Firebase

API

Firebase

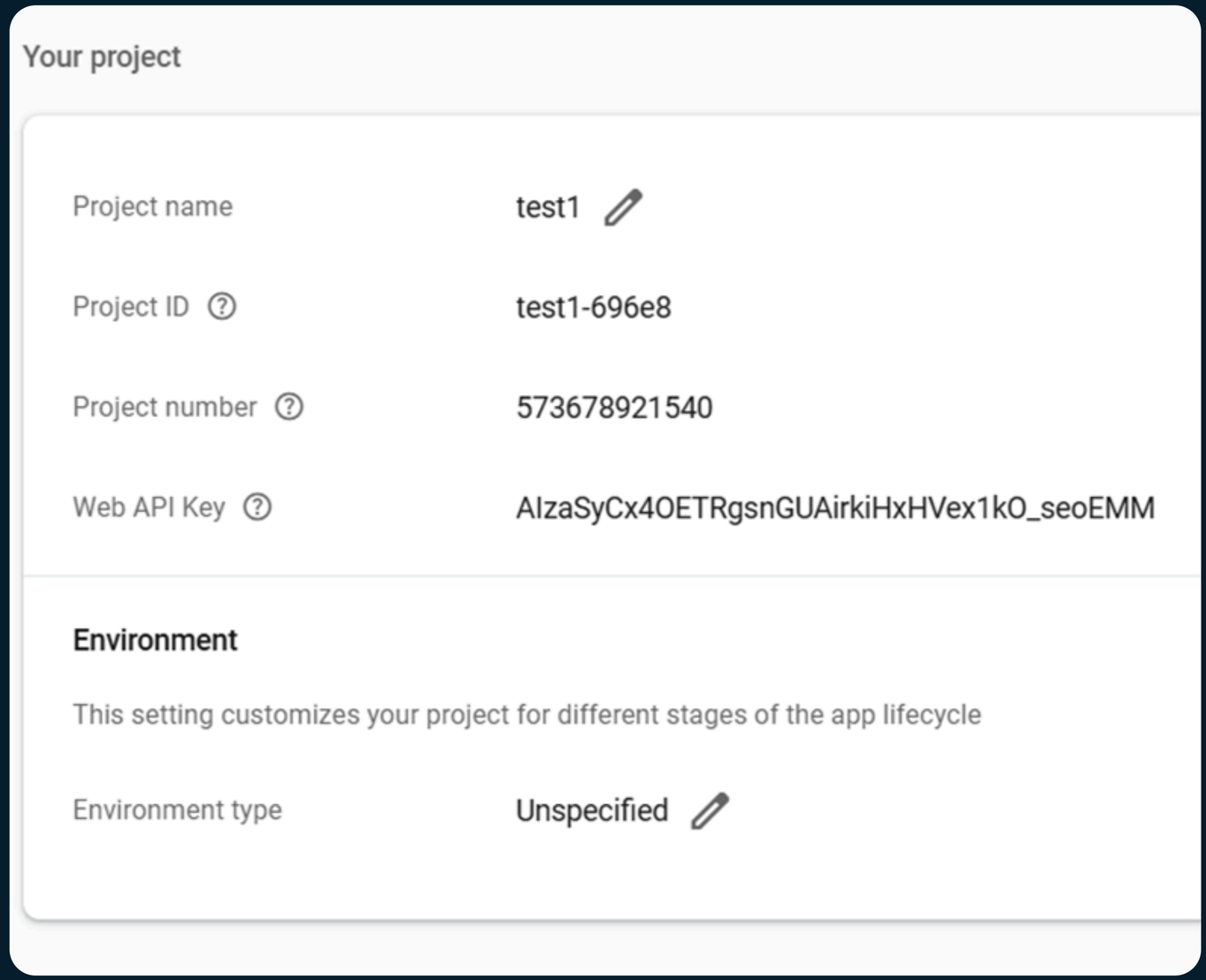
- Realtime Database, Authentication, and Cloud Functions for secure data handling and notifications.

Kivy

- For mobile app development and user interface design.

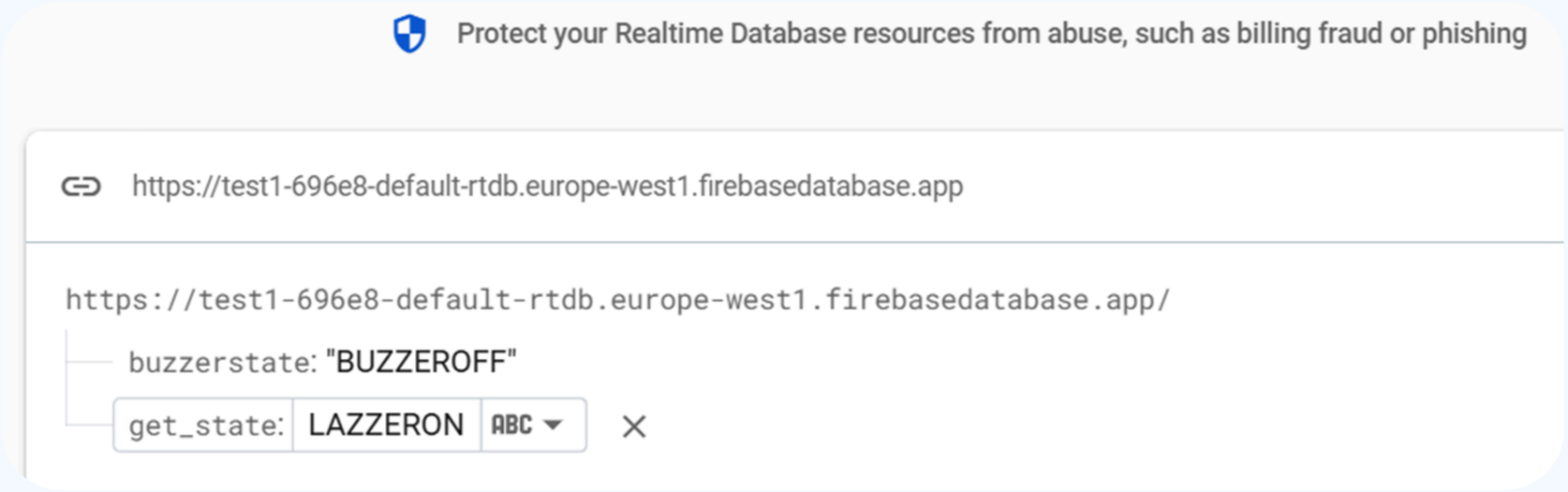
Libraries and APIs

- ESP32/8266 Firebase library for communication with the cloud.



WI-FI CONNECTION

- **It is assumed that the system will always have access to a stable Wi-Fi network for communication between the ESP32 microcontroller, Firebase, and the mobile app.**
- **Any interruptions in the network could cause delays or failures in real-time data synchronization and control commands.**



Integrating Firebase with Esp via an API make us able to:

User Authentication:

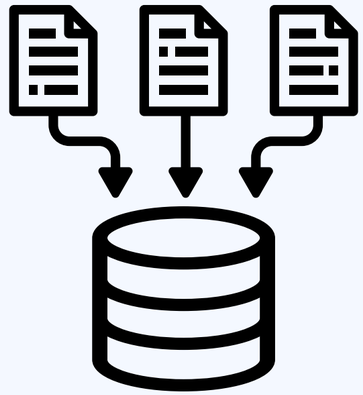
- Securely manage user access to control the laser alarm through Firebase Authentication.

Real-time Updates:

- Provide instant feedback and control for our laser alarm system

Send Data :

- Log real-time data such as laser sensor status or alarm activation to the Firebase Realtime Database.



Receive Data:

- Retrieve and act on commands sent from the Firebase console or the mobile application.

Admin SDK configuration snippet

Node.js Java Python Go

```
import firebase_admin
from firebase_admin import credentials

cred = credentials.Certificate("path/to/serviceAccountKey.json")
firebase_admin.initialize_app(cred)
```

Generate new private key

• Real-Time Notification System

Feature:

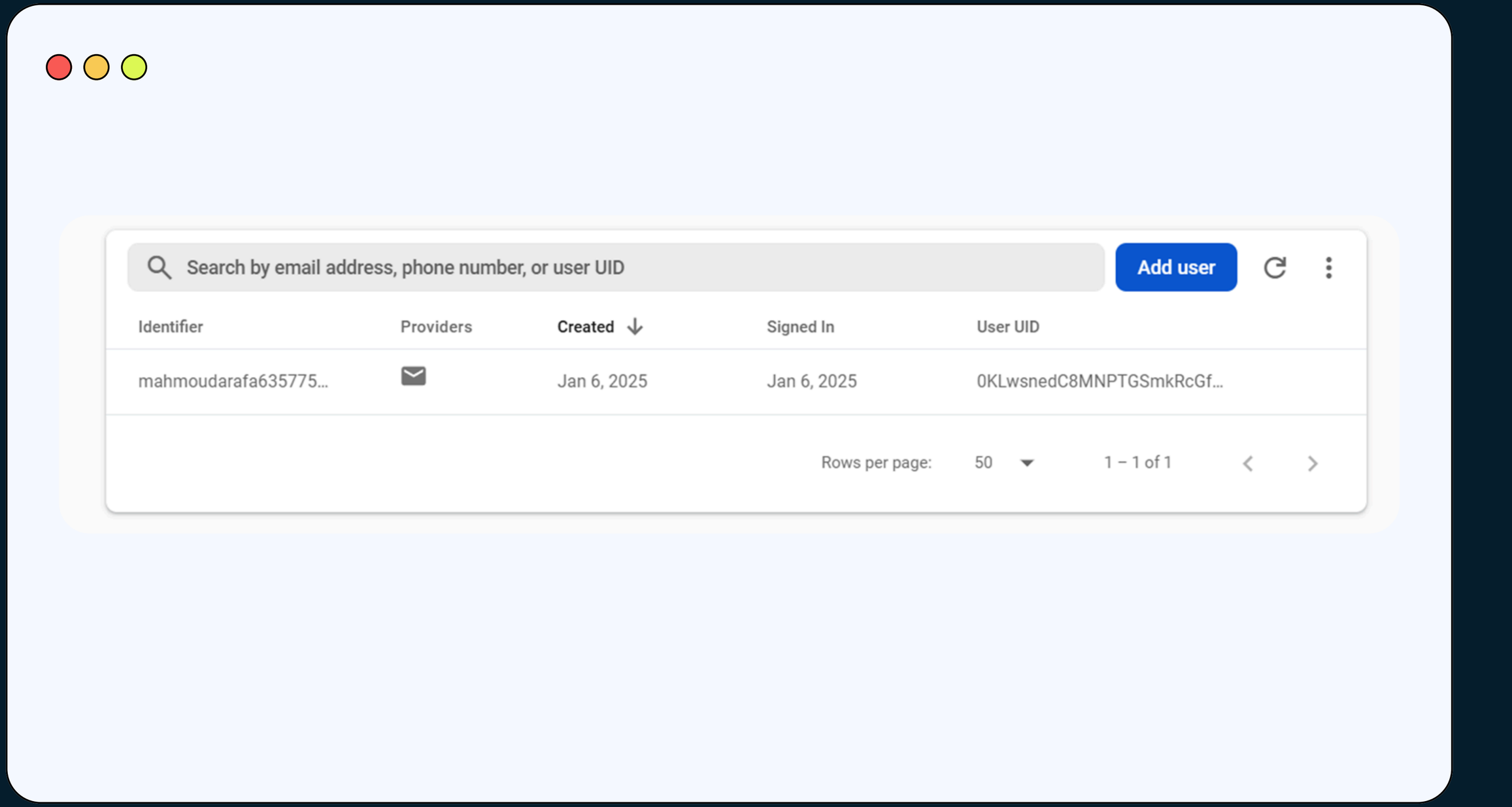
- The system should send real-time notifications for important events.

Functionality:

The system will send notifications when:

1. The gate is opened successfully (via the correct password or admin control).
2. A breach or unauthorized access is detected (invalid password entry or laser breach).
3. Someone entered without entering the password or an admin opened the gate.

- **These notifications will be sent to the mobile app so that users and admins can monitor and respond to security events remotely.**


Search by email address, phone number, or user UID
Add user
Identifier
Providers
Created
Signed In
User UID
mahmoudarafa635775...
Jan 6, 2025
Jan 6, 2025
0KLwsnedC8MNPTGSmkRcGf...
Rows per page:
50
1 – 1 of 1

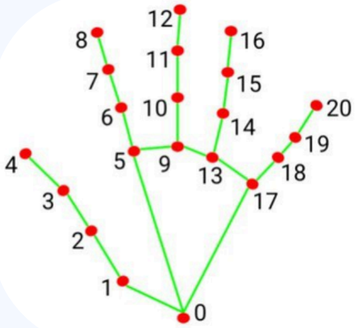
Computer vision using mediapipe

Mediapipe is a powerful open-source library developed by Google that provides ready-to-use machine learning solutions for real-time perception tasks such as face detection, hand tracking, pose estimation, and more.

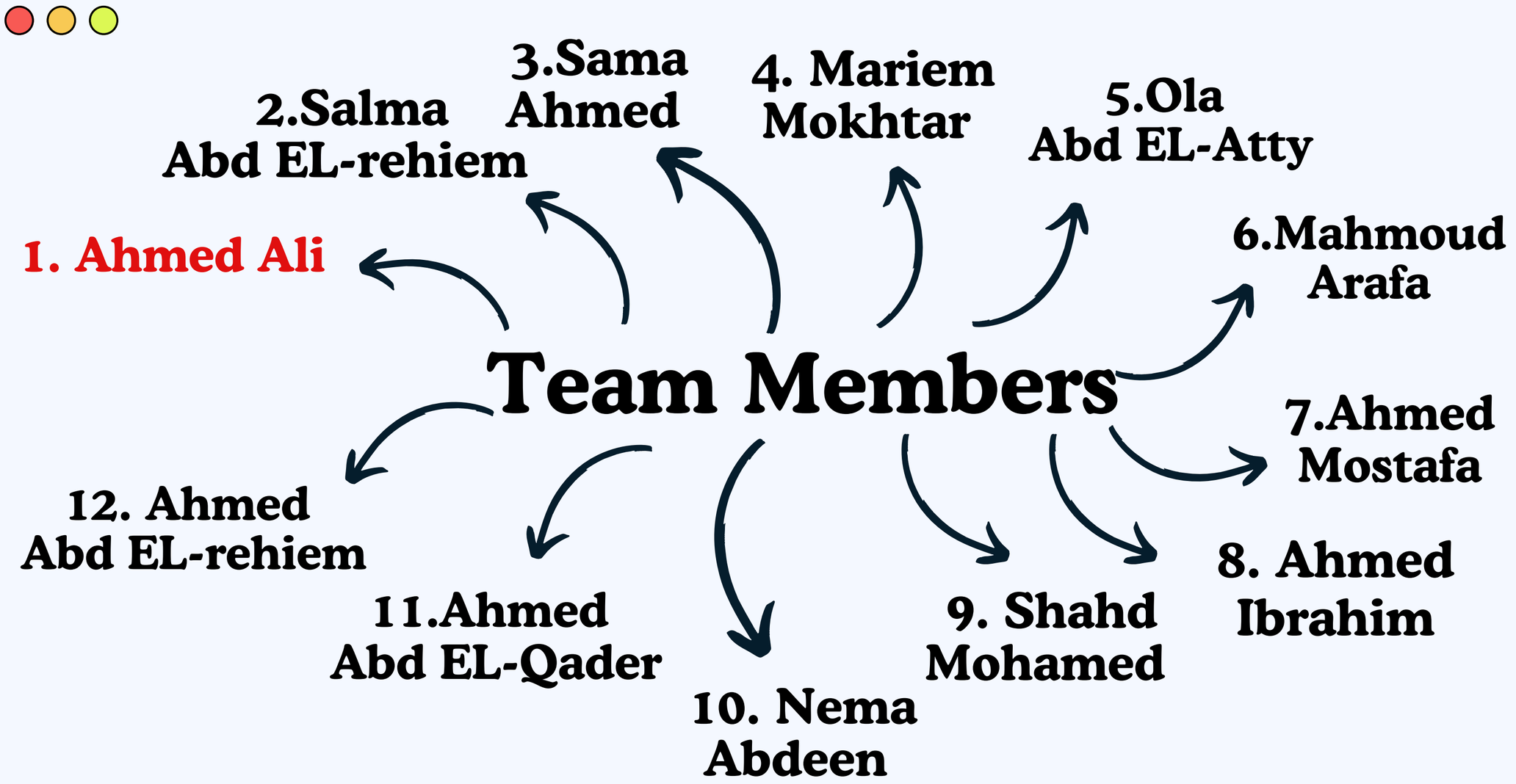
- It is widely used for computer vision tasks and supports multiple platforms, including Python, C++, and JavaScript.

Key Modules of Mediapipe:

- Hands.
- Face Detection.
- Face Mesh.
- Pose.
- Objectron.
- Holistic.
- Hair Segmentation.



0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP


Team Members
1. Ahmed Ali
2.Salma
Abd EL-rehiem
3.Sama
Ahmed
4. Mariem
Mokhtar
5.Ola
Abd EL-Atty
6.Mahmoud
Arafa
7.Ahmed
Mostafa
8. Ahmed
Ibrahim
9. Shahd
Mohamed
10. Nema
Abdeen
11.Ahmed
Abd EL-Qader
12. Ahmed
Abd EL-rehiem

Kivy | Protect | Data | Threat | Security | Attack | Firewall | Firebase

THANKYOU

Stay Safe, Stay Secure

Group”1"

Embedded System